UNIVERSIDAD POLITÉCNICA INTERNACIONAL

Técnicas de Programación

Entregable #1

Planificación del Proyecto #1

Sistema de gestión fitness para el gimnasio

C# (Windows Forms)

Estudiante	Denzell Valverde Jimenez
Profesor	Luis Felipe Mora Umaña
Fecha de entrega	8 de noviembre de 2026

Índice

1. Introducción **2**

1.1 Objetivo general . . . 2

1.2 Objetivos específicos . . . 2

2. Decisiones de diseño **3**

2.1 Patrón MVC . . . 3

2.2 Conceptos que se van a aplicar . . . 3

2.3 Organización del repositorio . . . 3

2.4 Uso de Jira y Git . . . 4

3. División del proyecto **5**

Epic 1: Sistema de gestión fitness para el gimnasio . . . 5

Feature 1.1: Estructura del proyecto y archivos de datos . . . 5

Feature 1.2: Gestión de usuarios . . . 6

Feature 1.3: Gestión de rutinas . . . 7

Feature 1.4: Clases del gimnasio y visitas . . . 8

Feature 1.5: Insignias . . . 8

Feature 1.6: Timeline de notificaciones . . . 9

Feature 1.7: Pestañas de información . . . 10

Feature 1.8: Estadísticas y reportes . . . 10

Feature 1.9: Calidad, documentación y entrega . . . 11

4. Cronograma (Diagrama de Gantt) **13**

5. Bibliografía **14**

6. Conclusiones y análisis de aprendizaje **14**

1. Introducción

Este documento es el Entregable #1 del Proyecto #1 de Técnicas de Programación. Aquí se planifica un sistema para llevar el control fitness de la rutina de ejercicios de los usuarios de un gimnasio. El sistema permitirá registrar usuarios, manejar rutinas y ver estadísticas.

El programa se va a hacer en C# con Windows Forms, usando programación orientada a objetos, el patrón MVC y buenas prácticas de Clean Code. Como es la primera iteración, los datos se van a cargar desde archivos CSV o JSON. El código se debe hacer pensando en que el Proyecto #2 va a continuar sobre esta base y puede traer cambios.

Para organizar el trabajo se dividió el proyecto en un Epic (el proyecto completo), Features (las funcionalidades) y PBIs (elementos de trabajo más pequeños) con sus tareas. También se incluye un diagrama de Gantt con las fechas propuestas.

1.1 Objetivo general

Planificar el desarrollo del sistema de gestión fitness de forma ordenada para poder cumplir con lo que pide el proyecto.

1.2 Objetivos específicos

- Dividir el proyecto en Epic, Features, PBIs y tareas.
- Definir cómo se va a organizar el programa con el patrón MVC.
- Elegir un patrón de diseño y los conceptos de POO que se van a usar.
- Crear un cronograma con un diagrama de Gantt.
- Definir cómo se va a organizar el repositorio y el tablero de Jira.

2. Decisiones de diseño

2.1 Patrón MVC

Se va a usar el patrón Modelo-Vista-Controlador para separar el programa en tres partes. La vista son las ventanas de Windows Forms. El controlador recibe lo que hace el usuario y decide qué hacer. El modelo son las clases con los datos, como Persona, Usuario y Rutina. Para leer y guardar los archivos se va a usar la interfaz IDataHandler con la clase FileHandler, así las capas quedan separadas y se pueden cambiar más adelante. El esquema parte del dibujo que se hizo en clase.

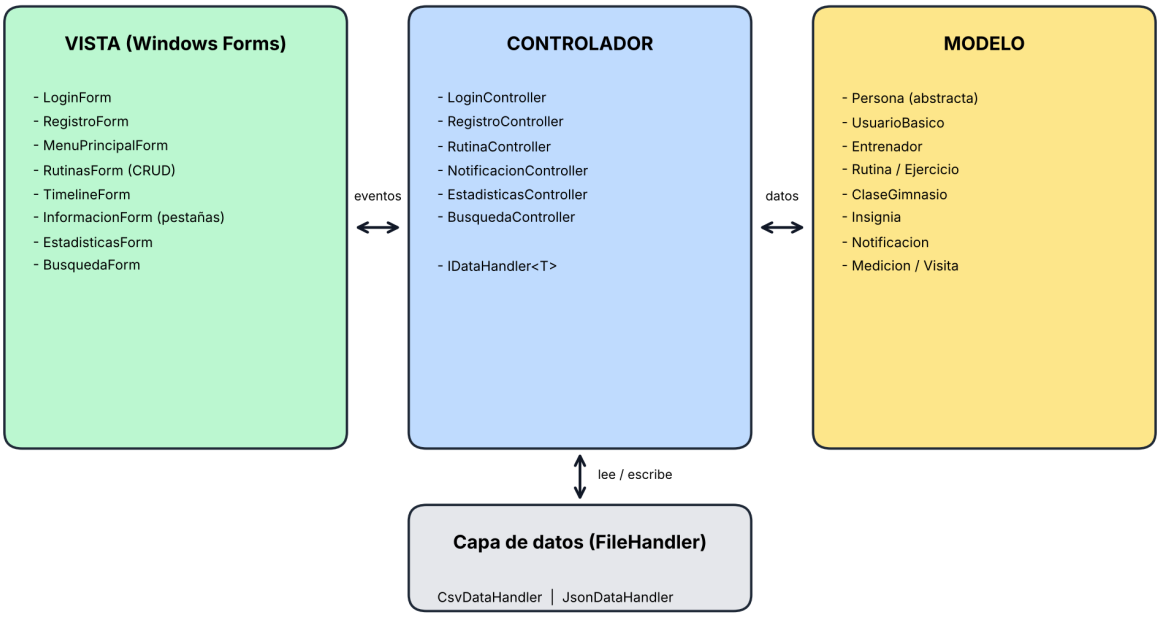


Figura 1. Esquema MVC del proyecto.

2.2 Conceptos que se van a aplicar

Concepto	Dónde se va a usar
Herencia	Persona es la clase padre de Usuario y Entrenador; ClaseGimnasio es el padre de yoga, crossfit, zumba y spinning.
Polimorfismo	Los distintos tipos de entrenador y de clases comparten métodos pero cada uno los hace a su manera.
Encapsulamiento	Los atributos son privados y se acceden con propiedades que validan los datos.
Patrón de diseño	Repository (IDataHandler y FileHandler) para separar la lectura de archivos del resto del programa. Se evaluará usar también Observer para las notificaciones.
SOLID y Clean Code	Nombres claros, métodos cortos que hagan una sola cosa, comentarios XML y manejo de excepciones.

2.3 Organización del repositorio

El repositorio se organiza como se vio en clases. La versión final estará en la rama master.

Elemento	Contenido
src	El código fuente (proyectos de Modelos, Controladores y Vistas y los archivos de datos).
docs	La documentación: este entregable y el PDF de documentación técnica.
.gitignore	Archivos que no se suben a Git (bin, obj, .vs).
README.md	Descripción del proyecto y cómo ejecutarlo.
Archivo .sln	La solución de Visual Studio.

2.4 Uso de Jira y Git

En Jira se va a crear el Epic, los Features y los PBIs con sus tareas, y se va a ir cambiando su estado mientras se avanza. En Git se van a hacer commits frecuentes con mensajes claros y una rama por funcionalidad, que luego se une a master.

3. División del proyecto

La tabla muestra cómo quedó dividido el proyecto. Después se explica cada PBI.

Código	Elemento	Tipo	PBIs
1	Sistema de gestión fitness para el gimnasio	Epic	25
1.1	Estructura del proyecto y archivos de datos	Feature	3
1.2	Gestión de usuarios	Feature	4
1.3	Gestión de rutinas	Feature	3
1.4	Clases del gimnasio y visitas	Feature	2
1.5	Insignias	Feature	2
1.6	Timeline de notificaciones	Feature	3
1.7	Pestañas de información	Feature	3
1.8	Estadísticas y reportes	Feature	2
1.9	Calidad, documentación y entrega	Feature	3

Epic 1: Sistema de gestión fitness para el gimnasio

Sistema en C# (Windows Forms) para que los usuarios de un gimnasio lleven el control de sus rutinas, mediciones, visitas, insignias y estadísticas, y para que los entrenadores administren las rutinas, con datos guardados en archivos CSV o JSON.

Feature 1.1: Estructura del proyecto y archivos de datos

Consiste en crear la estructura del programa con el patrón MVC y hacer que el sistema pueda leer y guardar información en archivos CSV o JSON.

PBI 1.1.1: Estructura de la solución y del repositorio	
Descripción	Como estudiante, se necesita crear la solución en Visual Studio con las carpetas del proyecto (Modelos, Controladores y Vistas) y el repositorio en GitHub organizado como se vio en clases.
Criterio de Aceptación	La solución abre y compila sin errores, y el repositorio tiene las carpetas src y docs, el README y el .gitignore, con la versión final en la rama master.
Tareas	• Crear el repositorio en GitHub y clonarlo. • Crear la solución y los proyectos en Visual Studio. • Agregar el README y el .gitignore. • Hacer el primer commit y subirlo a master.

PBI 1.1.2: Lectura y guardado de archivos (IDataHandler y FileHandler)	
Descripción	Como estudiante, se necesita una interfaz IDataHandler y una clase FileHandler que lean y guarden los datos en archivos CSV o JSON, para que más adelante se pueda cambiar la forma de guardar datos sin tocar el resto del programa.
Criterio de Aceptación	Los controladores usan la interfaz y no el archivo directamente; si el archivo no existe o tiene errores, se muestra un mensaje y el programa no se cierra.
Tareas	• Crear la interfaz IDataHandler. • Crear FileHandler para CSV y para JSON. • Manejar los errores de lectura y escritura con try-catch.

PBI 1.1.3: Archivos de datos de ejemplo	
Descripción	Como estudiante, se necesitan archivos con datos de ejemplo de usuarios, entrenadores, rutinas, clases, mediciones y visitas para poder probar el sistema.
Criterio de Aceptación	Cada archivo tiene al menos 40 registros y la mayoría de los usuarios tiene al menos 5 rutinas.
Tareas	• Definir las columnas de cada archivo. • Crear los datos de ejemplo. • Revisar que los identificadores coincidan entre archivos.

Feature 1.2: Gestión de usuarios

Incluye los dos tipos de usuario (usuario básico y entrenador), el registro, el inicio de sesión y la búsqueda.

PBI 1.2.1: Clases Persona, Usuario y Entrenador	
Descripción	Como estudiante, se necesita crear la clase Persona y que Usuario y Entrenador hereden de ella. El usuario tiene nombre, usuario, edad, peso, estatura, IMC, rutinas e insignias. Los entrenadores pueden ser personales o del gimnasio.
Criterio de Aceptación	El IMC se calcula solo a partir del peso y la estatura, y los datos tienen validaciones (por ejemplo, edad y peso mayores que cero).
Tareas	• Crear la clase Persona. • Crear Usuario con el cálculo del IMC. • Crear Entrenador y sus dos tipos. • Agregar comentarios XML a las clases.

PBI 1.2.2: Registro de usuarios	
Descripción	Como usuario, se necesita una ventana para registrarme en la aplicación del gimnasio.
Criterio de Aceptación	La ventana revisa que los campos no estén vacíos, que el nombre de usuario no exista y que edad y peso sean válidos; después guarda al usuario en el archivo.
Tareas	• Crear la ventana de registro. • Crear el RegistroController. • Validar los datos y guardar.

PBI 1.2.3: Inicio de sesión	
Descripción	Como usuario o entrenador, se necesita iniciar sesión para entrar al sistema según mi tipo de usuario.
Criterio de Aceptación	Si los datos son correctos se abre el menú principal del rol; si no, se muestra un mensaje de error.
Tareas	• Crear la ventana de login. • Crear el LoginController. • Abrir el menú según el rol.

PBI 1.2.4: Búsqueda de usuarios	
Descripción	Como entrenador, se necesita buscar usuarios por nombre o nombre de usuario.
Criterio de Aceptación	La búsqueda muestra los resultados y avisa cuando no encuentra nada.
Tareas	• Crear el BusquedaController. • Crear la ventana de búsqueda. • Agregarla al menú principal.

Feature 1.3: Gestión de rutinas

Permite agregar, editar, modificar y eliminar rutinas y asignarlas a los usuarios.

PBI 1.3.1: Clases Rutina y Ejercicio	
Descripción	Como estudiante, se necesita crear las clases Rutina y Ejercicio, con series, repeticiones, peso, fecha de inicio y fecha de vencimiento.
Criterio de Aceptación	Una rutina tiene uno o más ejercicios y pertenece a un usuario.
Tareas	• Crear las clases. • Relacionarlas con Usuario y Entrenador. • Guardarlas en archivo.

PBI 1.3.2: Agregar, editar, modificar y eliminar rutinas	
Descripción	Como entrenador, se necesita una ventana para agregar, editar, modificar y eliminar rutinas.
Criterio de Aceptación	Cada acción valida los datos, pide confirmación al eliminar y actualiza el archivo y la pantalla.
Tareas	• Crear el RutinaController. • Crear la ventana con la lista y el formulario. • Validar los datos.

PBI 1.3.3: Asignar rutinas a usuarios	
Descripción	Como entrenador, se necesita asignar una rutina a un usuario y saber cuándo vence.
Criterio de Aceptación	La rutina queda asignada, el usuario recibe una notificación y se puede ver cuáles rutinas están por vencer.
Tareas	• Crear la asignación de rutina. • Calcular si la rutina está por vencer. • Enviar la notificación de rutina nueva.

Feature 1.4: Clases del gimnasio y visitas

Incluye las clases del gimnasio (yoga, hora de gimnasio abierto, crossfit, zumba y spinning) y el registro de visitas.

PBI 1.4.1: Clases del gimnasio	
Descripción	Como estudiante, se necesita crear la clase ClaseGimnasio y una clase hija por cada tipo de clase.
Criterio de Aceptación	Existen las cinco clases pedidas, con horario, cupo y entrenador.
Tareas	• Crear ClaseGimnasio. • Crear las clases hijas (yoga, crossfit, zumba, spinning, gimnasio abierto). • Cargarlas desde archivo.

PBI 1.4.2: Registro de visitas	
Descripción	Como usuario, se necesita registrar mis visitas al gimnasio.
Criterio de Aceptación	Cada visita guarda el usuario, la fecha y la hora, y no se permiten fechas futuras.
Tareas	• Crear la clase Visita. • Crear la opción de registrar visita. • Validar la fecha.

Feature 1.5: Insignias

Incluye las insignias por cumplir metas (musculatura, grasa y visitas) y las insignias de la vergüenza.

PBI 1.5.1: Insignias por metas cumplidas	
Descripción	Como usuario, se necesita ganar insignias cuando cumplo mis metas de musculatura, grasa o visitas al gimnasio.
Criterio de Aceptación	La insignia se asigna una sola vez y genera una notificación.
Tareas	• Crear la clase Insignia. • Revisar si el usuario cumplió cada meta. • Mostrar las insignias del usuario.

PBI 1.5.2: Insignias de la vergüenza	
Descripción	Como sistema, se necesita dar insignias de la vergüenza a quienes tienen menos visitas (top 10), no agregan su progreso o no cumplen sus objetivos.
Criterio de Aceptación	Las insignias se calculan en un rango de fechas y generan su mensaje en el timeline.
Tareas	• Calcular el top 10 de menos visitas. • Revisar quién no agregó progreso. • Asignar la insignia y notificar.

Feature 1.6: Timeline de notificaciones

Cada usuario y cada entrenador tiene su propia lista (timeline) de notificaciones.

PBI 1.6.1: Notificaciones del usuario	
Descripción	Como usuario, se necesita recibir mensajes como: has ganado la insignia X, nueva rutina agregada, descansa esta semana (al llegar a 5 visitas en la misma semana) y los mensajes de la vergüenza.
Criterio de Aceptación	Los mensajes se guardan con fecha y hora y se muestran del más reciente al más antiguo.
Tareas	• Crear la clase Notificacion. • Generar cada tipo de mensaje. • Guardar las notificaciones en archivo.

PBI 1.6.2: Notificaciones del entrenador	
Descripción	Como entrenador, se necesita saber cuándo van a expirar las rutinas y cuándo están cerca las mediciones.
Criterio de Aceptación	El aviso aparece algunos días antes de la fecha límite en el timeline del entrenador.
Tareas	• Revisar rutinas por vencer. • Revisar mediciones cercanas. • Crear el aviso.

PBI 1.6.3: Ventana del timeline	
Descripción	Como usuario o entrenador, se necesita una ventana para ver mis notificaciones.
Criterio de Aceptación	La ventana muestra la lista y se abre desde el menú principal.
Tareas	• Crear la ventana. • Crear el NotificacionController. • Agregarla al menú.

Feature 1.7: Pestañas de información

Son las pestañas donde el usuario consulta su historial.

PBI 1.7.1: Historial de visitas y últimas 5 rutinas	
Descripción	Como usuario, se necesita ver mi historial de visitas y mis últimas 5 rutinas.
Criterio de Aceptación	Las visitas aparecen de la más reciente a la más antigua y se muestran exactamente 5 rutinas (o menos, si no hay más).
Tareas	• Crear la ventana con pestañas. • Mostrar el historial de visitas. • Mostrar las últimas 5 rutinas.

PBI 1.7.2: Historial de mediciones	
Descripción	Como usuario, se necesita guardar y ver mis mediciones (peso, grasa, musculatura e IMC).
Criterio de Aceptación	Cada medición se guarda con su fecha y el IMC se recalcula con el peso.
Tareas	• Crear la clase Medicion. • Crear la opción para registrar medición. • Mostrar el historial en una pestaña.

PBI 1.7.3: Control de aumentos de rutina	
Descripción	Como usuario, se necesita ver cuánto han aumentado los pesos de mis rutinas.
Criterio de Aceptación	La pestaña compara los pesos entre rutinas y muestra si subieron o bajaron.
Tareas	• Calcular la diferencia entre rutinas. • Mostrarla en la pestaña.

Feature 1.8: Estadísticas y reportes

Las estadísticas se calculan cuando se abre la pantalla (no se guardan), dentro de fechas específicas y solo para el usuario básico.

PBI 1.8.1: Estadísticas de progreso	
Descripción	Como usuario, se necesita ver el aumento de mis pesos y mi mejora en los objetivos dentro de un rango de fechas.
Criterio de Aceptación	Se muestran los datos del rango elegido; si la fecha inicial es mayor que la final, se muestra un error.
Tareas	• Crear la clase que calcula el progreso. • Agregar el filtro de fechas. • Mostrar los resultados.

PBI 1.8.2: Top 10 de visitantes y de ausentes	
Descripción	Como usuario, se necesita ver el top 10 de quienes más visitan el gimnasio y el top 10 de los más ausentes.
Criterio de Aceptación	Los dos listados se ordenan bien, respetan las fechas y solo los ve el usuario básico.
Tareas	• Contar visitas por usuario. • Ordenar y tomar los primeros 10. • Crear la ventana de reportes y agregarla al menú.

Feature 1.9: Calidad, documentación y entrega

Son las tareas para revisar el código, documentar el proyecto y entregarlo.

PBI 1.9.1: Manejo de excepciones	
Descripción	Como estudiante, se necesita manejar los errores con try-catch en todas las capas para que el programa no se cierre.
Criterio de Aceptación	Los errores esperados muestran un mensaje claro al usuario.
Tareas	• Revisar los lugares donde puede haber errores. • Agregar try-catch y mensajes. • Probar con datos incorrectos.

PBI 1.9.2: Revisión con Sonar Analyzer	
Descripción	Como estudiante, se necesita revisar el código con Sonar Analyzer para seguir las buenas prácticas.
Criterio de Aceptación	No quedan errores graves y se corrigen los problemas principales que reporta.
Tareas	• Instalar Sonar Analyzer. • Corregir lo que reporte. • Revisar nombres, métodos cortos y comentarios.

PBI 1.9.3: Documentación y entrega en Moodle	
Descripción	Como estudiante, se necesita el PDF de documentación técnica, el tablero de Jira actualizado y el archivo .txt para Moodle.
Criterio de Aceptación	El PDF tiene portada, índice, introducción, decisiones de diseño, desarrollo, análisis de resultados y conclusiones; el .txt tiene el enlace a Jira y al repositorio.
Tareas	• Escribir el PDF de documentación técnica. • Actualizar Jira y el README. • Revisar que la versión final esté en master. • Subir el .txt a Moodle.

4. Cronograma (Diagrama de Gantt)

Primero se hace la base (estructura, archivos y usuarios), después las funcionalidades y, al mismo tiempo, la documentación y la revisión del código. Las fechas después de este entregable son una propuesta y se pueden ajustar a la fecha oficial de entrega.

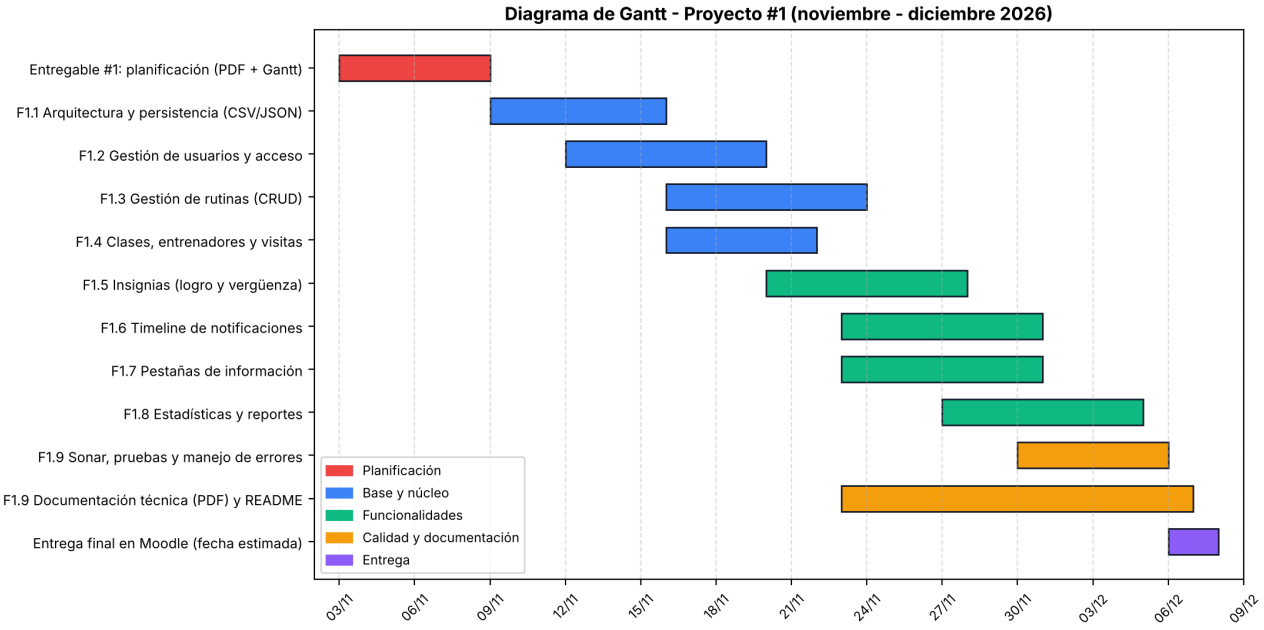


Figura 2. Diagrama de Gantt del proyecto.

Hito	Fecha
Entregable #1 (planificación)	Domingo 8 de noviembre de 2026
Base del sistema lista (1.1 a 1.4)	23 de noviembre de 2026
Funcionalidades listas (1.5 a 1.8)	4 de diciembre de 2026
Entrega final en Moodle	Fecha por confirmar (estimada: 6 de diciembre de 2026)

5. Bibliografía

• Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.

• Microsoft. (s. f.). Documentación de Windows Forms y C#. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/desktop/winforms/

• Atlassian. (s. f.). Jira Software. https://www.atlassian.com/software/jira

• Material y ejemplos vistos en clase de Técnicas de Programación (2026).

6. Conclusiones y análisis de aprendizaje

Con esta planificación se pudo dividir un enunciado grande en partes más pequeñas y fáciles de manejar: un Epic, nueve Features y 25 PBIs con sus tareas. Esto ayuda a ver por dónde empezar, y se notó que primero hay que hacer la estructura y los archivos de datos porque todo lo demás depende de eso.

Se aprendió que el programa debe hacerse pensando en cambios futuros, por eso se separó el acceso a los archivos con IDataHandler. También se aprendió a usar el formato de Epic, Feature y PBI y a hacer un diagrama de Gantt para repartir el tiempo.

Lo que más puede costar es unir todas las partes al final y corregir lo que reporte Sonar Analyzer. Para evitarlo se va a subir código a Git con frecuencia y se va a revisar el código poco a poco y no solo al terminar.